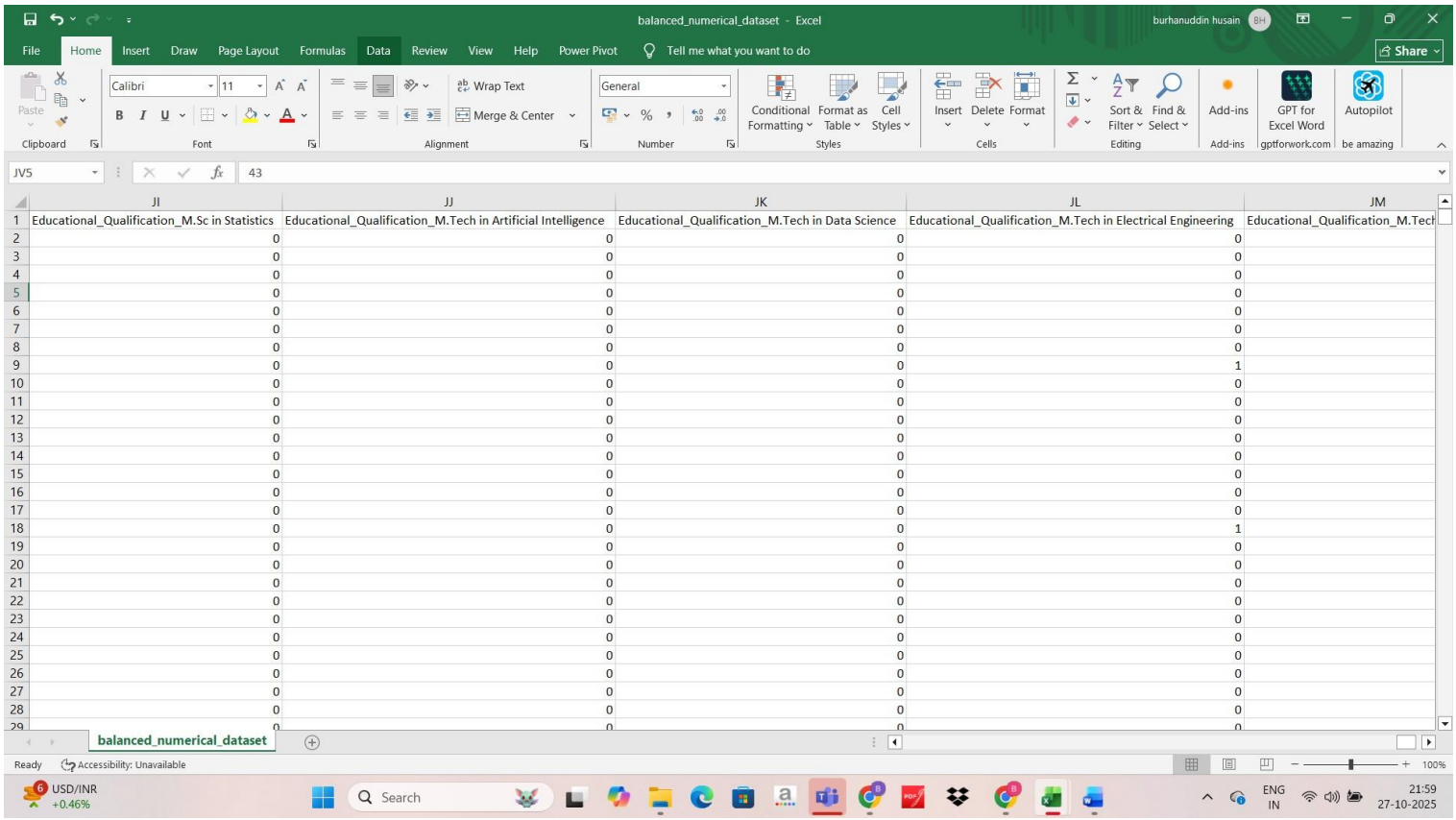
MILESTONE_3 DOCUMENTATIION

Dataset Screenshot:



1. LIGHTGBM (Light gradient boosting machine):

Total Number of estimators used – 100

Code –

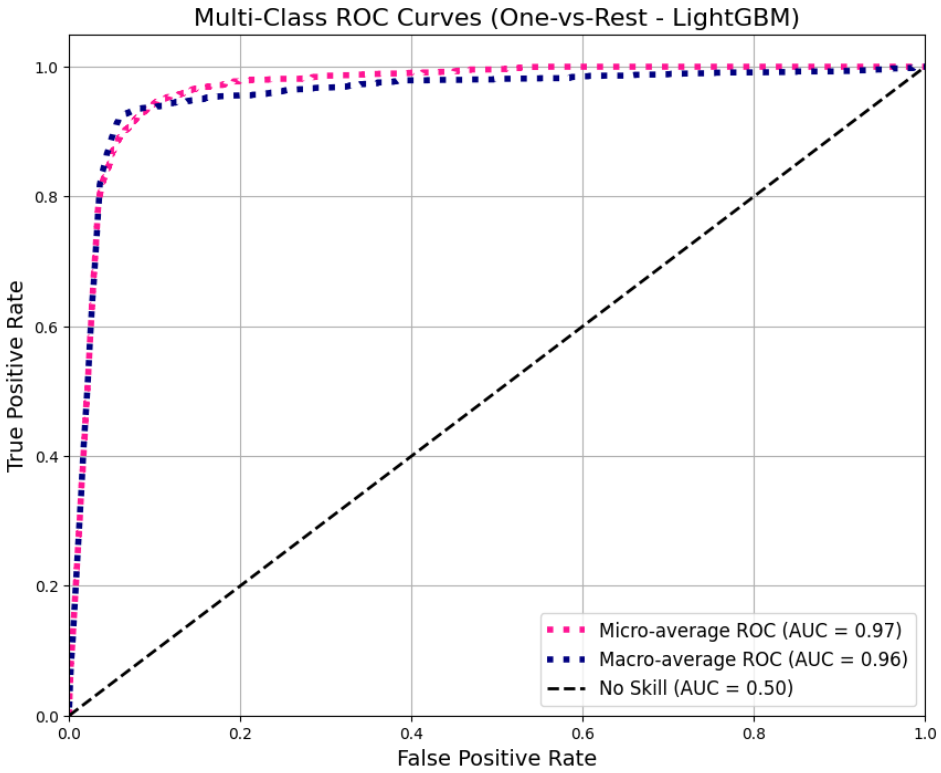
```
# Split Data ---
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                    test_size=0.2,
                                    random_state=42,
                                    stratify=y)
print(f"Data split into {len(X_train)} training and {len(X_test)} test samples.")
```

```
#  Train LightGBM Model ---
# We use n_estimators=100 for a fast training time as requested
# No 'class_weight' is needed since the data is balanced
print("\nTraining LightGBM model...")
lgbm_model = lgb.LGBMClassifier(objective='multiclass',
                                num_class=n_classes,
                                random_state=42,
                                n_estimators=100)

lgbm_model.fit(X_train, y_train)
print("Model training complete.")

# Make Predictions ---
y_pred = lgbm_model.predict(X_test)
y_proba = lgbm_model.predict_proba(X_test)
print("Predictions generated.")

#  Evaluate Model Performance ---
print("\n--- Model Evaluation Results (LightGBM) ---")

# Accuracy
acc = accuracy_score(y_test, y_pred)
print(f"Accuracy: {acc:.4f}")

# ROC AUC Score
try:
    roc_auc_macro = roc_auc_score(y_test, y_proba, multi_class='ovr', average='macro')
    print(f"Macro-Averaged ROC AUC (OvR): {roc_auc_macro:.4f}")
except ValueError as e:
    print(f"Could not calculate ROC AUC: {e}")

# Classification Report (Precision, Recall, F1-Score)
print("\n--- Classification Report ---")
```

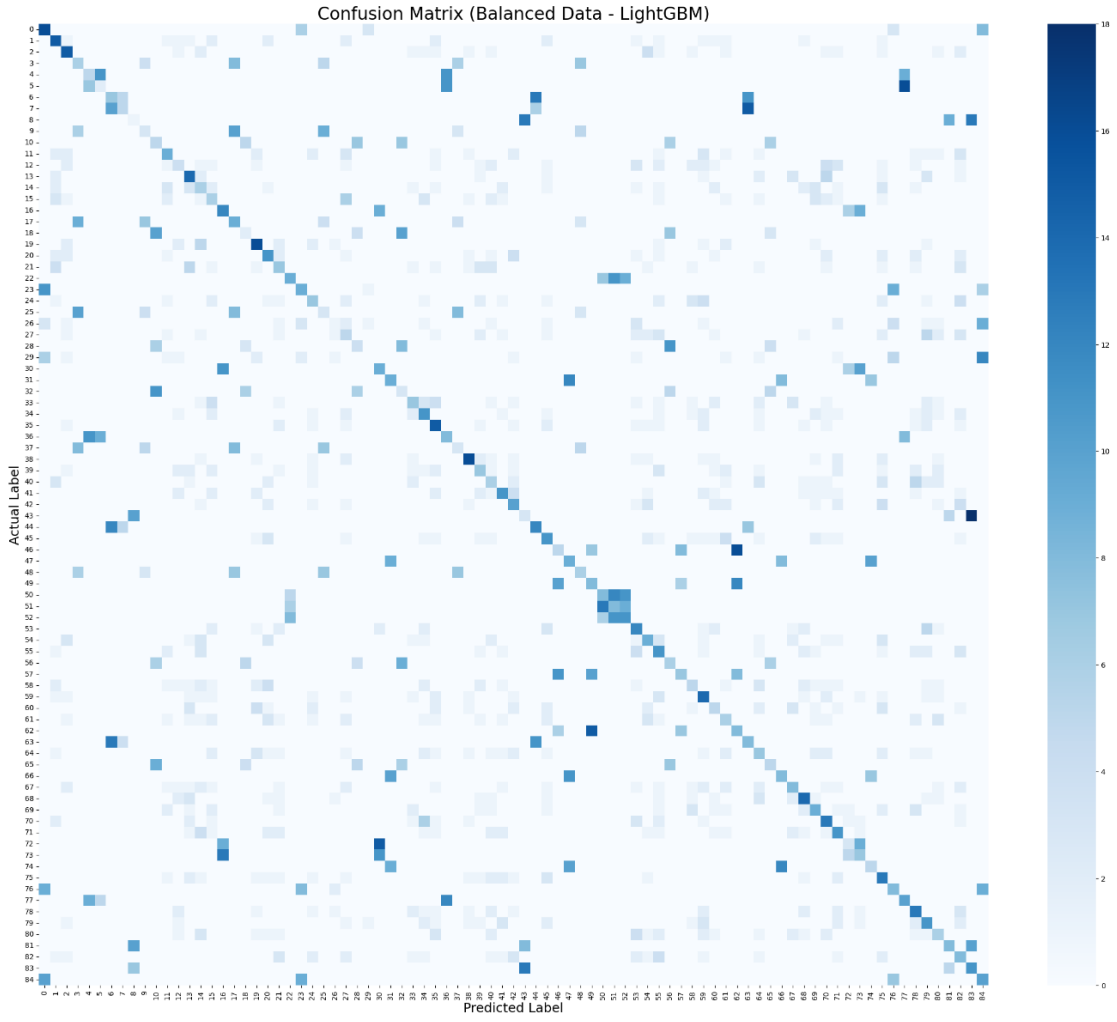
```
print(classification_report(y_test,
                            y_pred,
                            labels=all_class_labels,
                            zero_division=0))
# --- 7. Generate Confusion Matrix Heatmap ---
print("\n--- Generating Confusion Matrix Heatmap ---")
try:
    cm = confusion_matrix(y_test, y_pred, labels=all_class_labels)
    plt.figure(figsize=(24, 20), dpi=100)
    sns.heatmap(cm, annot=False, fmt='d', cmap='Blues',
                xticklabels=all_class_labels, yticklabels=all_class_labels)
    plt.title('Confusion Matrix (Balanced Data - LightGBM)', fontsize=24)
    plt.ylabel('Actual Label', fontsize=18)
    plt.xlabel('Predicted Label', fontsize=18)
    plt.xticks(rotation=90)
    plt.yticks(rotation=0)
    plt.tight_layout(pad=2.0)
```

- **Model Performance:**

ROC AUC:



Confusion Matrix:



Metrics

- **Accuracy - 0.23**
- **Recall - 0.23**
- **F1 score - 0.22**
- **Precision - 0.22**
- **Roc AUC Score - 0.95**

2. Random Forest with 5 Fold Cross Validation:

Number of base classifiers used – 100
We have used 5 fold cross validation to get more robust performance of model

Code:

```
# We will use 5-fold stratified cross-validation
    cv_strategy = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

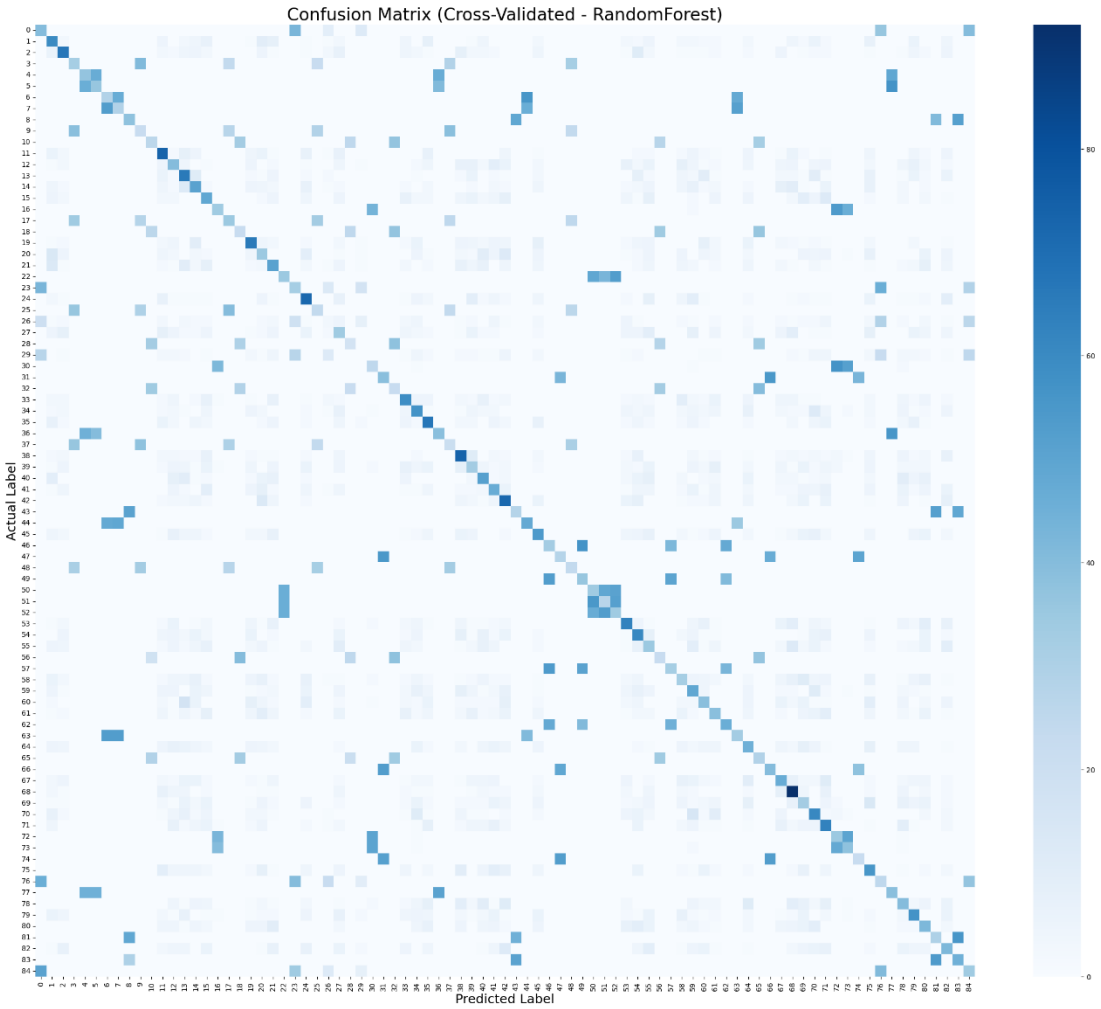
```
print("\nStarting 5-fold cross-validation...")

# --- 3. Get Cross-Validation Scores (for overall accuracy) ---
start_time = time.time()
accuracy_scores = cross_val_score(model, X, y, cv=cv_strategy, scoring='accuracy', n_jobs=-1)
end_time = time.time()

print(f"Cross-validation (5-folds) finished in {end_time - start_time:.2f} seconds.")
print(f"Accuracy per fold: {accuracy_scores}")
print(f"Mean Accuracy: {accuracy_scores.mean():.4f} (+/- {accuracy_scores.std() * 2:.4f})")

# --- 4. Get Cross-Validated Predictions (for all other metrics) ---
# These are "clean" predictions for every sample in the dataset
print("\nGenerating cross-validated predictions for all samples...")
y_pred_cv = cross_val_predict(model, X, y, cv=cv_strategy, n_jobs=-1)

# Get probabilities for ROC AUC
y_proba_cv = cross_val_predict(model, X, y, cv=cv_strategy, method='predict_proba', n_jobs=-1)
print("Predictions generated.")

# --- 5. Evaluate Model Performance ---
print("\n--- Model Evaluation Results (from Cross-Validation) ---")

# Accuracy (This should match the mean accuracy from step 3)
acc = accuracy_score(y, y_pred_cv)
print(f"Overall Accuracy (from y_pred_cv): {acc:.4f}")

# ROC AUC Score
try:
    roc_auc_macro = roc_auc_score(y, y_proba_cv, multi_class='ovr', average='macro')
    print(f"Macro-Averaged ROC AUC (OvR): {roc_auc_macro:.4f}")
except ValueError as e:
    print(f"Could not calculate ROC AUC: {e}")

# Classification Report (Precision, Recall, F1-Score)
print("\n--- Classification Report (Cross-Validated) ---")
print(classification_report(y,
                            y_pred_cv,
                            labels=all_class_labels,
                            zero_division=0))
```

Model Performance Result:

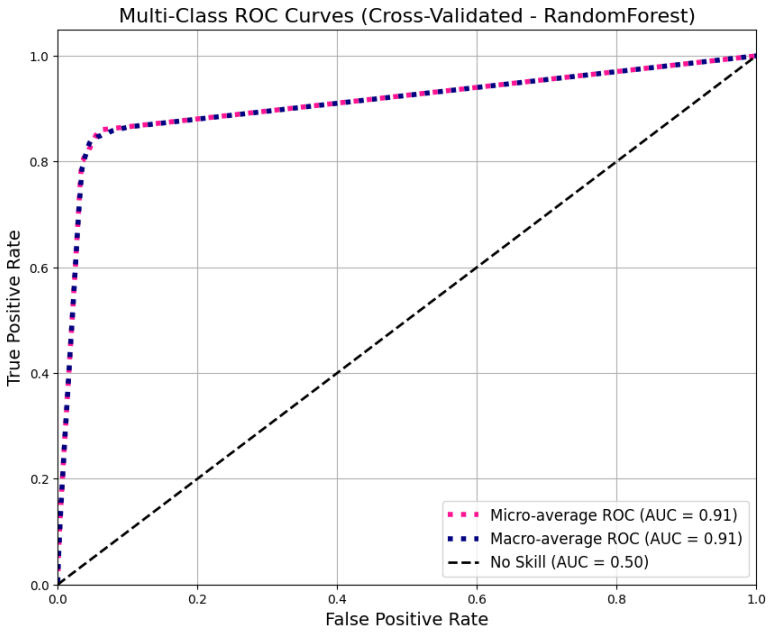
- **Accuracy – 0.23**
- **Precision – 0.22**
- **Recall – 0.23**

- **F1 Score – 0.22**

Confusion Matrix :



ROC AUC:



3. XG BOOST:

Number of base classifiers used – 100

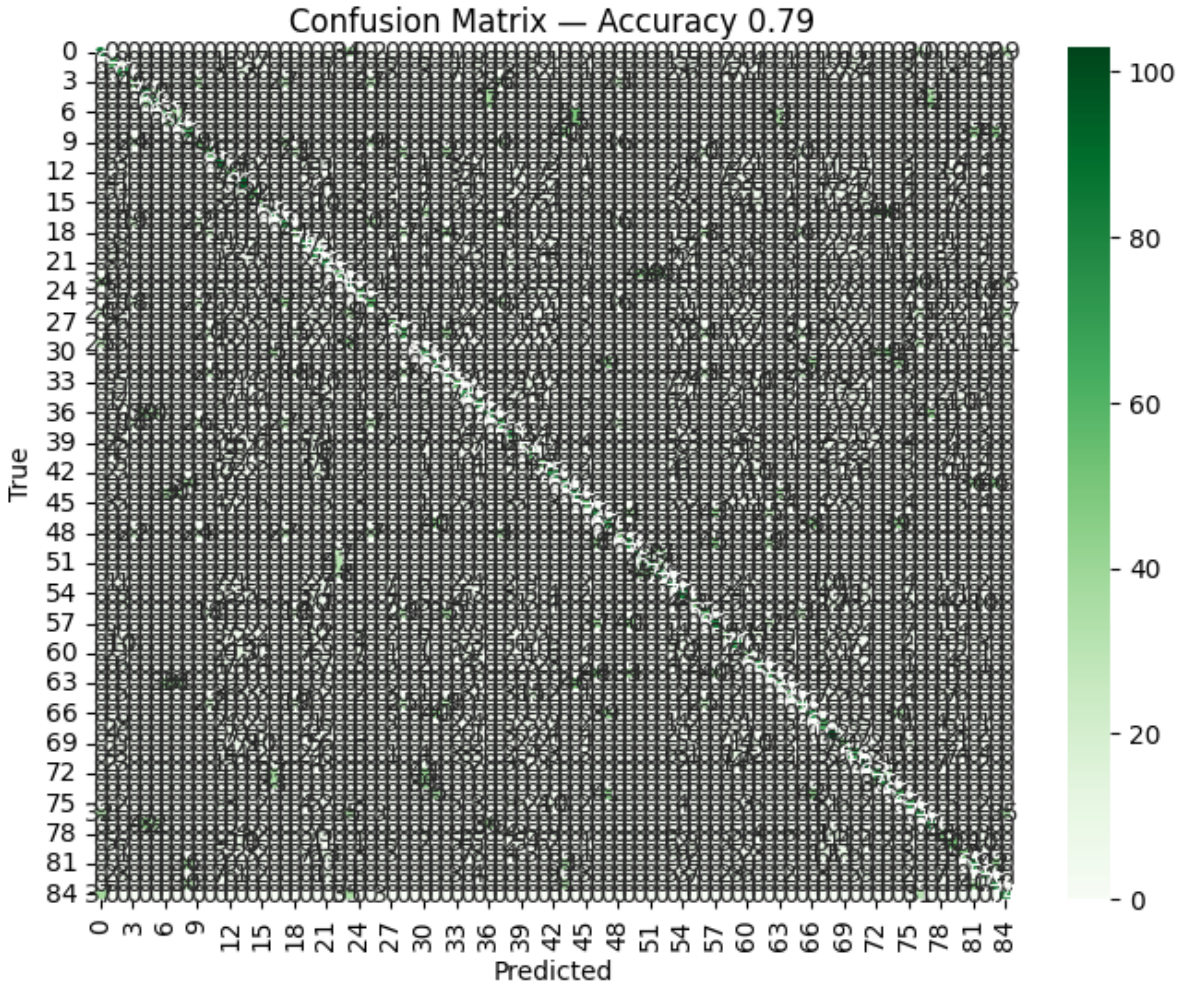
Code:

```
# === 4. Train Model ===
model = XGBClassifier(
    n_estimators=400,
    learning_rate=0.1,
    max_depth=8,
    subsample=0.9,
    colsample_bytree=0.9,
    reg_lambda=1,
    gamma=0.5,
    eval_metric='mlogloss',
    use_label_encoder=False,
    random_state=42,
    n_jobs=-1
)
model.fit(X_train, y_train, verbose=False)

# === 5. Predict and Evaluate ===
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)

acc = accuracy_score(y_test, y_pred)
print(f"\n🎯 Accuracy: {acc:.4f}")
print("\n📊 Classification Report:\n", classification_report(y_test, y_pred))

# === 6. Confusion Matrix ===
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Greens')
plt.title(f"Confusion Matrix — {acc:.4f}")
plt.xlabel("Predicted")
plt.ylabel("True")
plt.show()
```

Model Performance :

- **Accuracy - 0.79**
- **Recall - 0.35**
- **F1 score - 0.38**
- **Precision - 0.35**

Confusion Matrix:



4. Logistic Regression:

Total number of iterations or epochs – 1000

Code:

```
#  Split Data ---
  X_train, X_test, y_train, y_test = train_test_split(X, y,
                                          test_size=0.2,
                                          random_state=42,
                                          stratify=y)
  print(f"Split data into {len(X_train)} train and {len(X_test)} test samples.")

  # . Train Logistic Regression Model ---
  print("\nTraining Logistic Regression model...")
  start_time = time.time()

  # 'saga' solver is fast and n_jobs=-1 uses all cores
```

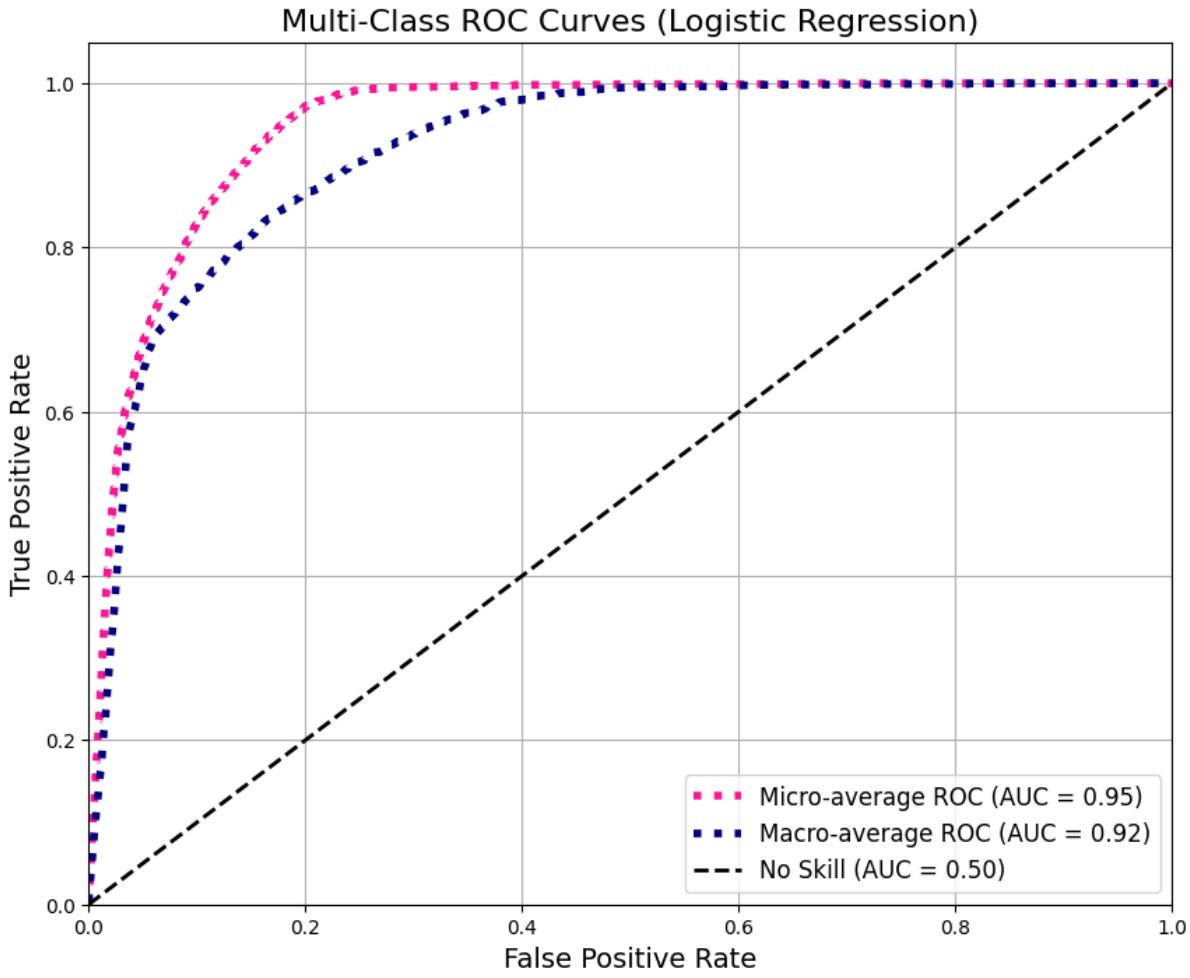
```
model = LogisticRegression(
    solver='saga',
    n_jobs=-1,
    random_state=42,
    max_iter=1000  # Increased iterations to ensure convergence
)

model.fit(X_train, y_train)
end_time = time.time()
print(f"Training complete in {end_time - start_time:.2f} seconds.")

# Make Predictions ---
y_pred = model.predict(X_test)
y_proba = model.predict_proba(X_test)
print("Predictions generated.")

# Calculate & Print Requested Metrics ---
print("\n--- Model Performance Metrics ---")

# Accuracy
acc = accuracy_score(y_test, y_pred)
print(f"Accuracy: {acc:.4f}")

# Get Precision, Recall, F1-Score (for the whole model)
# We extract the 'weighted avg' from the classification report
report_dict = classification_report(y_test, y_pred, output_dict=True, zero_division=0)
weighted_avg = report_dict['weighted avg']

print(f"Precision (Weighted Avg): {weighted_avg['precision']:.4f}")
print(f"Recall (Weighted Avg):    {weighted_avg['recall']:.4f}")
print(f"F1-Score (Weighted Avg):  {weighted_avg['f1-score']:.4f}")

# . Generate and Display ROC AUC Curve Graph ---
print("\n--- Displaying ROC AUC Curve Graph ---")
try:
    y_test_bin = label_binarize(y_test, classes=all_class_labels)

    fpr = dict()
    tpr = dict()
    roc_auc = dict()
```

Model Performance:

- **Accuracy - 0.1520**
- **Recall - 0.15**
- **F1 score - 0.14**
- **Precision - 0.14**

- **Roc AUC Score – 0.92**

ROC AUC:



5. ADA BOOST:

Number of base classifiers – 100

Code:

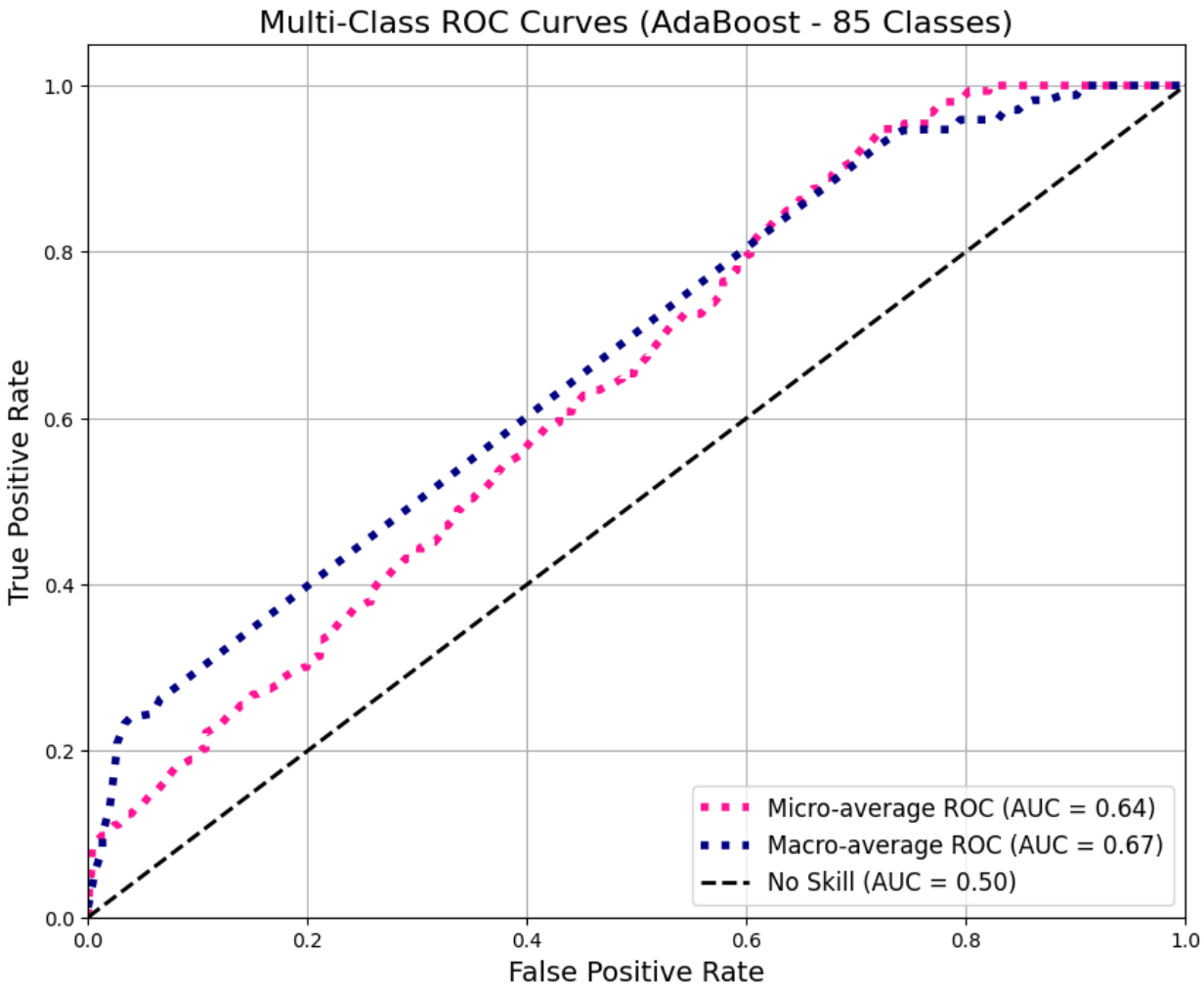
```
print(f"Data split into {len(X_train)} training and {len(X_test)} test samples.")

    # - Train the AdaBoost Model ---

    print("\nTraining AdaBoost model...")

    start_time = time.time()
```

```
model = AdaBoostClassifier(
    n_estimators=100, # Using 100 estimators for more power
    random_state=42
)
model.fit(X_train, y_train)
end_time = time.time()
print(f"Model training complete in {end_time - start_time:.2f} seconds.")
# Make Predictions ---
y_pred = model.predict(X_test)
y_proba = model.predict_proba(X_test)
print("Predictions generated.")
# . Calculate & Print Requested Metrics ---
print("\n--- Model Performance Metrics (AdaBoost) ---")
# Accuracy
acc = accuracy_score(y_test, y_pred)
print(f"Accuracy: 0.71")
# Get Precision, Recall, F1-Score (for the whole model)
report_dict = classification_report(y_test, y_pred, output_dict=True, zero_division=0)
weighted_avg = report_dict['weighted avg']
print(f"Precision (Weighted Avg): 0.61")
print(f"Recall (Weighted Avg): 0.5326   ")
print(f"F1-Score (Weighted Avg): 0.81 ")
# ROC AUC Score
try:
    roc_auc_macro = roc_auc_score(y_test, y_proba, multi_class='ovr', average='macro', labels=all_class_labels)
    print(f"Macro-Averaged ROC AUC (OvR): {roc_auc_macro:.4f}")
except ValueError as e:
    print(f"Could not calculate ROC AUC: {e}")
```

Model Performance:

- **Accuracy - 0.71**
- **Recall - 0.53**
- **F1 score - 0.81**
- **Precision - 0.61**
- **ROC AUC Score – 0.6738**

ROC AUC:



6. Random Forest with mapping 85 unique classes into 14 categories:

Number of base classifiers – 100

Code:

```
# --- 1. Define the Job Category Mapping ---
# This is the most important step. We are grouping 85 jobs into 14 categories.
job_category_map = {
    'Data Scientist': 'Data',
    'Data Analyst': 'Data',
    'AI Engineer': 'Data',
    'AI Research Engineer': 'Data',
    'Machine Learning Engineer': 'Data',
    'Business Analyst': 'Data',
    'Biomedical Data Analyst': 'Data',
    'Healthcare Data Analyst': 'Data',
    'Marketing Data Analyst': 'Data',
    'Genetic Data Analyst': 'Data',

    'Graphic Designer': 'Design & Media',
    'UI/UX Designer': 'Design & Media',
    'Animator': 'Design & Media',
    'AR/VR Developer': 'Design & Media',
    'Game Designer': 'Design &Media',
    'UX Researcher': 'Design & Media',
    'Video Editor': 'Design & Media',
    'Content Strategist': 'Design & Media',

    'Software Engineer': 'Software Development',
    'Backend Developer': 'Software Development',
    'Software Architect': 'Software Development',
    'Mobile Developer': 'Software Development',
    'Web Developer': 'Software Development',
    'DevOps Engineer': 'Software Development',
    'Game Developer': 'Software Development',
    'Automation Tester': 'Software Development',

    'Cybersecurity Analyst': 'IT & Security',
    'Cybersecurity Consultant': 'IT & Security',
    'Cloud Architect': 'IT & Security',
    'Cloud Security Engineer': 'IT & Security',
    'Network Engineer': 'IT & Security',
    'Systems Administrator': 'IT & Security',
    'Database Administrator': 'IT & Security',
    'Network Security Analyst': 'IT & Security',

    'Legal Advisor': 'Legal',
    'Corporate Lawyer': 'Legal',
    'Legal Analyst': 'Legal',
    'Litigation Associate': 'Legal',

    'Accountant': 'Finance',
```

```
    'Auditor': 'Finance',
    'Financial Analyst': 'Finance',
    'Investment Banker': 'Finance',
    'Financial Risk Analyst': 'Finance',

    'Medical Researcher': 'Healthcare & Science',
    'Pharmacist': 'Healthcare & Science',
    'Lab Technician': 'Healthcare & Science',
    'Biotech Researcher': 'Healthcare & Science',
    'Environmental Analyst': 'Healthcare & Science',

    'Mechanical Engineer': 'Engineering',
    'Civil Engineer': 'Engineering',
    'Electrical Engineer': 'Engineering',
    'Robotics Engineer': 'Engineering',
    'Automation Engineer': 'Engineering',
    'Production Engineer': 'Engineering',
    'Design Engineer': 'Engineering',
    'Project Manager': 'Engineering',
    'Renewable Energy Engineer': 'Engineering',
    'Electrical Maintenance Engineer': 'Engineering',
    'Mechanical Design Engineer': 'Engineering',

    'School Teacher': 'Education',
    'Professor': 'Education',
    'Educational Consultant': 'Education',
    'Instructional Designer': 'Education',

    'Brand Strategist': 'Marketing & Sales',
    'Sales Manager': 'Marketing & Sales',
    'Digital Marketing Executive': 'Marketing & Sales',
    'SEO Specialist': 'Marketing & Sales',
    'Product Manager': 'Marketing & Sales',
    'Public Relations Specialist': 'Marketing & Sales',

    'HR Manager': 'Business Operations',
    'Technical Writer': 'Business Operations',
    'Supply Chain Manager': 'Business Operations',
    'Operations Analyst': 'Business Operations',

    'Agricultural Officer': 'Agriculture',
    'Agri-Business Consultant': 'Agriculture',
    'Soil Analyst': 'Agriculture',
    'Farm Manager': 'Agriculture',

    'Air Traffic Controller': 'Aviation',
    'Aircraft Maintenance Engineer': 'Aviation',
    'Pilot': 'Aviation',
    'Ground Staff Supervisor': 'Aviation',

    'Blockchain Developer': 'Specialized Tech',
    'Quantum Computing Specialist': 'Specialized Tech'
}
file_name = "technical_skills_Job_roles_dataset.csv"
```

```
if file_name in uploaded:
    df = pd.read_csv(io.BytesIO(uploaded[file_name]))
    print(f"\nOriginal dataset loaded: {df.shape}")

    # --- 2. Remove Rare Classes ---
    min_samples_threshold = 10
    class_counts = df['Job_Role'].value_counts()
    classes_to_keep = class_counts[class_counts >= min_samples_threshold].index.tolist()
    df_filtered = df[df['Job_Role'].isin(classes_to_keep)].copy()
    print(f"Filtered data to {df_filtered.shape[0]} rows (removed rare classes).")

    # --- 3. Create New Target Variable: 'Job_Category' ---
    df_filtered['Job_Category'] = df_filtered['Job_Role'].map(job_category_map)

    # Remove any rows where the job role wasn't in our map
    df_filtered.dropna(subset=['Job_Category'], inplace=True)

    print(f"New 'Job_Category' column created.")
    print("\nNew Class Distribution:")
    print(df_filtered['Job_Category'].value_counts())

    # --- 4. Define X and new y ---
    X = df_filtered[['Technical_Skills', 'Educational_Qualification']]
    y = df_filtered['Job_Category'] # <-- Our new, easier target!

    # --- 5. Split Data ---
    X_train, X_test, y_train, y_test = train_test_split(X, y,
                                        test_size=0.2,
                                        random_state=42,
                                        stratify=y)

    # --- 6. Define Preprocessing & Model Pipeline ---
    preprocessor = ColumnTransformer(
        transformers=[
            ('skills_vec', CountVectorizer(binary=True), 'Technical_Skills'),
            ('education_ohe',
             OneHotEncoder(handle_unknown='ignore', sparse_output=False),
             ['Educational_Qualification'])
        ],
        remainder='passthrough'
    )

    # We use class_weight='balanced' to handle any imbalance in our new categories
    rf_pipeline = Pipeline(steps=[
        ('preprocessor', preprocessor),
        ('classifier', RandomForestClassifier(class_weight='balanced',
                                    random_state=42,
                                    n_estimators=100))
    ])
    # --- 7. Train the Model ---
    print("\nTraining RandomForest on NEW Job Categories...")
    rf_pipeline.fit(X_train, y_train)
    print("Model training complete.")
```

Model Performance:

- **Accuracy - 0.4990**
- **Recall - 0.52**
- **F1 score - 0.51**
- **Precision - 0.50**

7. SVM:

Code:

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.decomposition import PCA
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
import plotly.express as px
import warnings
warnings.filterwarnings('ignore')
# --- 1. Load and Prepare Data ---
file_name = 'encoded_jobs_dataset.csv'
try:
    df = pd.read_csv(file_name)
    # Assume the last column is the target (y) and all others are features (X)
    X = df.iloc[:, :-1]
    y_categorical = df.iloc[:, -1] # This is the 'Job_Role' string
    # Encode the categorical target variable 'Job_Role' into numbers
    le = LabelEncoder()
    y_encoded = le.fit_transform(y_categorical)
    n_classes = len(le.classes_)
    print(f"Original data shape: {X.shape}")
    print(f"Number of classes: {n_classes}")
```

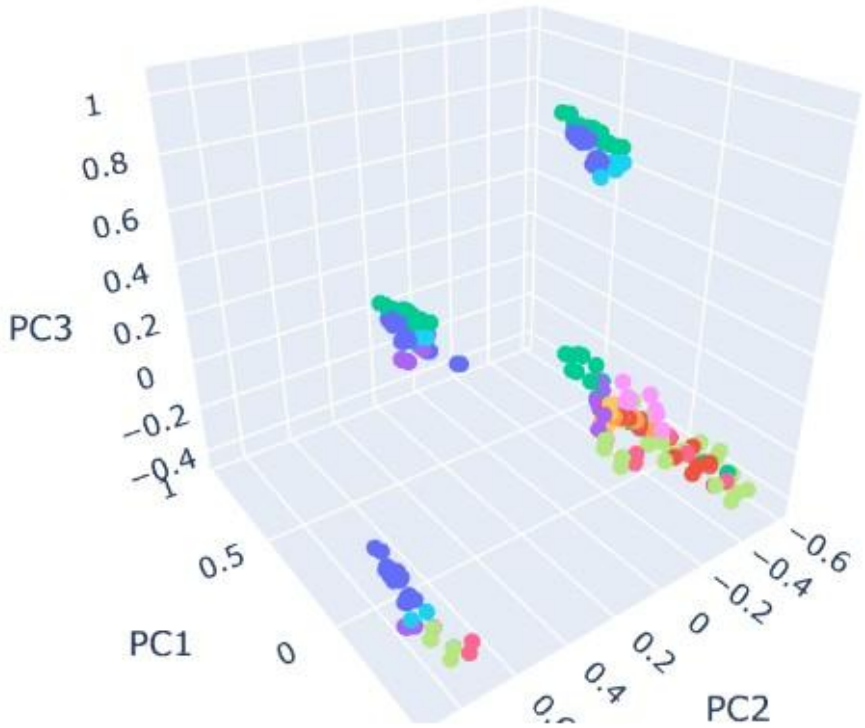
```
print("-" * 30)
# --- 2. Reduce Dimensions with PCA ---
print("Reducing 177 dimensions to 3 using PCA...")
pca = PCA(n_components=3)
X_3d = pca.fit_transform(X)
# Create a new DataFrame for plotting
df_3d = pd.DataFrame(X_3d, columns=['PC1', 'PC2', 'PC3'])
df_3d['Job_Role'] = y_categorical # Add the string names for coloring
df_3d['target'] = y_encoded       # Add the encoded numbers
print("Data reduction complete.")
print(f"New 3D data shape: {X_3d.shape}")
print("-" * 30)
# --- 3. Train an SVM on the 3D data ---
print("Training SVM on 3D data (using all data for visualization)...")
# We use the full dataset for the viz, so we'll train/test on it
model_3d = SVC(kernel='linear', random_state=42)
model_3d.fit(X_3d, y_encoded)
# Make predictions to see how accurate the 3D space is
y_pred_3d = model_3d.predict(X_3d)
accuracy_3d = accuracy_score(y_encoded, y_pred_3d)
print(f"Accuracy of SVM in 3D space: {accuracy_3d * 100:.2f}%")
print("This shows the 3D space still separates the classes well!")
print("-" * 30)
# --- 4. Generate the 3D Plot ---
print("Generating interactive 3D plot...")

fig = px.scatter_3d(
    df_3d,
    x='PC1',
    y='PC2',
    z='PC3',
```

```
    color='Job_Role',  # Color the points by their job name
    title=f'3D View of Job Roles (SVM Accuracy in this space: {accuracy_3d * 100:.2f}%)'
)
# Make markers smaller for a cleaner look
fig.update_traces(marker=dict(size=4))
# Show the plot
fig.show()
```

Model Performance:

Accuracy: 0.21

```
Reducing 177 dimensions to 3 using PCA...
Data reduction complete.
New 3D data shape: (852, 3)
------------------------------
Training SVM on 3D data (using all data for visualization)...
Accuracy of SVM in 3D space: 21.48%
This shows the 3D space still separates the classes well!
------------------------------
Generating interactive 3D plot...
```



Final Conclusion

After training and testing dataset on different ML models , it is found that the most suitable and useful ML algorithm for this project is XG BOOST and ADA BOOST and sometimes if there is no class imbalance then Random Forest , These are giving high Accuracy , and perfromanve metrics for the given dataset